Netflix Educational Content

Jonathan Garcia, Emma Whitehead, Augustine Perez, Alex Balido

CSCI3304 - Database Systems

Table of Contents

Our Goal

Our goal is to make it easy to find educational content on Netflix. With the large number of shows and movies on the platform along with recommended algorithms, top shows and new content being added daily much of the content can get buried and never watched. Our solution is to create a GUI that allows users to find the content they want and create a watchlist.

Database Design

Database Choice and Data Model

Since the project needed to just be stored locally we went with SQLite, a relational database (RDBMS) that was perfect for our use case. SQLite doesn't require a separate server and we were able to have the database stored in a file, in our case **netflix_db.db.** An added bonus is that SQLite is highly optimized for read operations which ensures a fast response time when filtering through our records.

The data model was structured in such a way that we could easily retrieve the data for the educational content use case. Our core table was **Netflix_IMDB** which combined data from multiple sources into a single structure, lessening the complexity when querying the database. Additional tables include **Subjects** which controlled the vocabulary for the dropdown menus (e.g.,Documentaries, Nature). **Watchlist** held the users data, this is where CRUD went to work to store the lists label as well as the teachers name. We used **Title_Subjects** to handle the many to many relationships that movies have with subjects, being in more than one category. We also used **Watchlist_Items** to store the actual movies inside a list.

CRUD Operations

While the application implements the full spectrum of Create,Read,Update,Delete, when it comes to CRUD operations the ones we rely on most heavily are the create and read operations. The **create** operation is used in the *Watchlist* feature, where you can define a subject and teacher name. The application executes an INSERT into the *Watchlist table* and generates a primary key for the list. **Read** is used with the WHERE clause to filter through our *Netflix_IMDB* table for things such as genre , age_rating etc. Our **update** feature is when you can add items to a specific watchlist, this utilizes insert statements into the *Watchlist_Items* with a *Movie_ID* to a *Watchlist_ID* to update the state of that specific watchlist. Finally we have **delete**, this is the most straight forward of the bunch, teachers can remove the entire watchlist or specific movies from their list that they may have watched or just want removed. This operation utilizes DELETE FROM in which the WHERE clause is equal to the id we wish to remove.

Back-End Analytical Queries

There are a total of 5 Back-End Queries:

Query 1: Find by academic Keyword (in Description): This allows user to locate media with a keyword that searches the detailed description.

```
if selected_option == "Query 1: Find by Academic Keyword (in Description)":
    keyword = param3_keyword_entry.get()
    rating = param3_rating_entry.get()

    if not keyword or not rating:
        messagebox.showwarning("Input Error", "Please enter a Keyword and a Min Rating.")
        conn.close()
        return

    try:
        keyword_param = f'%{keyword}%'
        params = (keyword_param, keyword_param, float(rating))

        query = """
            SELECT title, averageRating, type, description
            FROM Netflix_IMDB
            WHERE
                (title LIKE ? OR description LIKE ?)
                AND CAST(averageRating AS REAL) >= ?
            ORDER BY
                CAST(averageRating AS REAL) DESC
            LIMIT 20;
        """
```

Query 2: Find by Genre (Top Rated): This allows users to include Genre in their search and will list from the top rated first.

```
elif selected_option == "Query 2: Find by Genre (Top-Rated)":
    subject = param1_combo.get()
    rating = param1_rating_entry.get()

    if not subject or not rating:
        messagebox.showwarning("Input Error", "Please select a Subject and enter a Min Rating.")
        conn.close()
        return

    try:
        subject_param = f'%{subject}%'
        params = (subject_param, float(rating))

        query = """
            SELECT title, type, averageRating, release_year
            FROM Netflix_IMDB
            WHERE
                listed_in LIKE ?
                AND CAST(averageRating AS REAL) >= ?
            ORDER BY
                CAST(averageRating AS REAL) DESC
            LIMIT 20;
        """
```

Query 3: Find Films by Age Rating and Duration: This query allows users to include the age rating and duration.

```
elif selected_option == "Query 3: Find Films by Age Rating & Duration":
    age_rating = param2_age_entry.get()
    duration = param2_duration_entry.get()

    if not age_rating or not duration:
        messagebox.showwarning("Input Error", "Please enter an Age Rating and a Max Duration.")
        conn.close()
        return

    try:
        params = (age_rating, int(duration))

        query = """
            SELECT title, rating, duration, averageRating, description
            FROM Netflix_IMDB
            WHERE
                type = 'Movie'
                AND rating = ?
                AND CAST(REPLACE(duration, ' min', '') AS INTEGER) <= ?
            ORDER BY
                CAST(averageRating AS REAL) DESC
            LIMIT 20;
        """
```

Query 4: Find Average Rating Per Year: This was one of the extra queries that we thought might be helpful when looking for academic and educational content. This would show the average rating per movie and show per year.

```
elif selected_option == "Query 4: Find Average Rating Per Year":
    query = """
        SELECT
            n.release_year,
            ROUND(AVG(n.averageRating), 2) AS avg_rating,
            COUNT(*) AS title_count
        FROM Netflix_IMDB AS n
        WHERE n.averageRating IS NOT NULL
        GROUP BY n.release_year
        ORDER BY n.release_year DESC;
    """
```

Query 5: Find Average Best Movies Per Year: This was another extra query that we thought might be helpful when looking for academic and educational content. It would allow the user to see the best rated movies and shows per year.

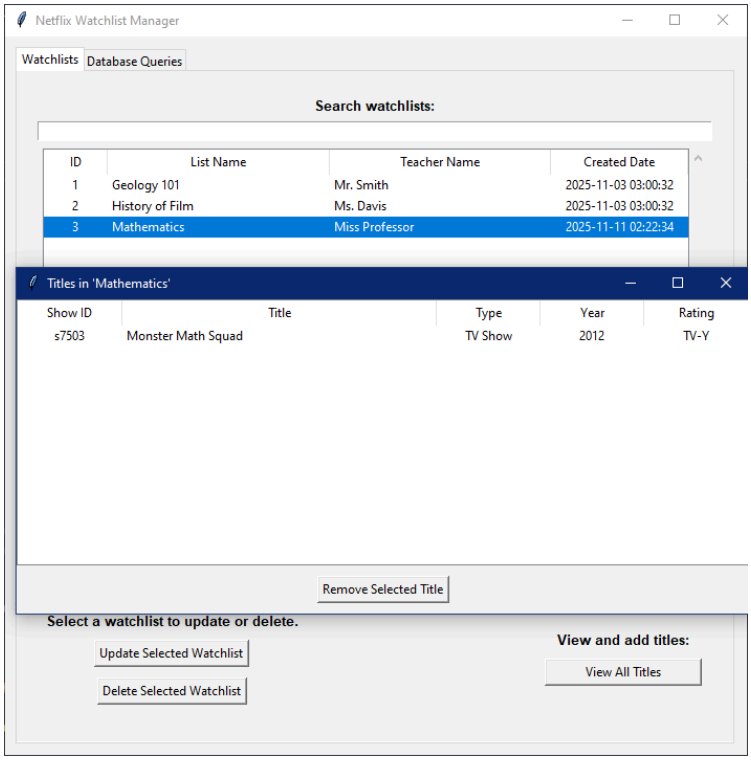
```
elif selected_option == "Query 5: Find Best Movies Per Year":
    query = """
        WITH RankedMovies AS (
            SELECT
                title,
                release_year,
                averageRating,
                numVotes,
                genres,
                ROW_NUMBER() OVER(
                    PARTITION BY release_year
                    ORDER BY averageRating DESC, numVotes DESC
                ) AS rank
            FROM Netflix_IMDB
            WHERE
                type = 'Movie'
                AND averageRating IS NOT NULL
                AND numVotes >= 1000
        )
        SELECT
            release_year,
            title,
            averageRating,
            numVotes,
            genres
        FROM RankedMovies
        WHERE rank = 1
        ORDER BY release_year DESC;
    """
```

Tkinter Documentation

The GUI for our project was implemented using tkinter, Python's standard library for GUIs. It provides interfaces for creating, viewing, updating, and deleting watchlists, as well as browsing all available titles and adding or removing titles from specific watchlists. The GUI includes search functionality, tables with scrollbars, right-click context menus, and detailed pop-up windows for title information. There are two tabs, one titled *Watchlists* and the other *Database Queries.*

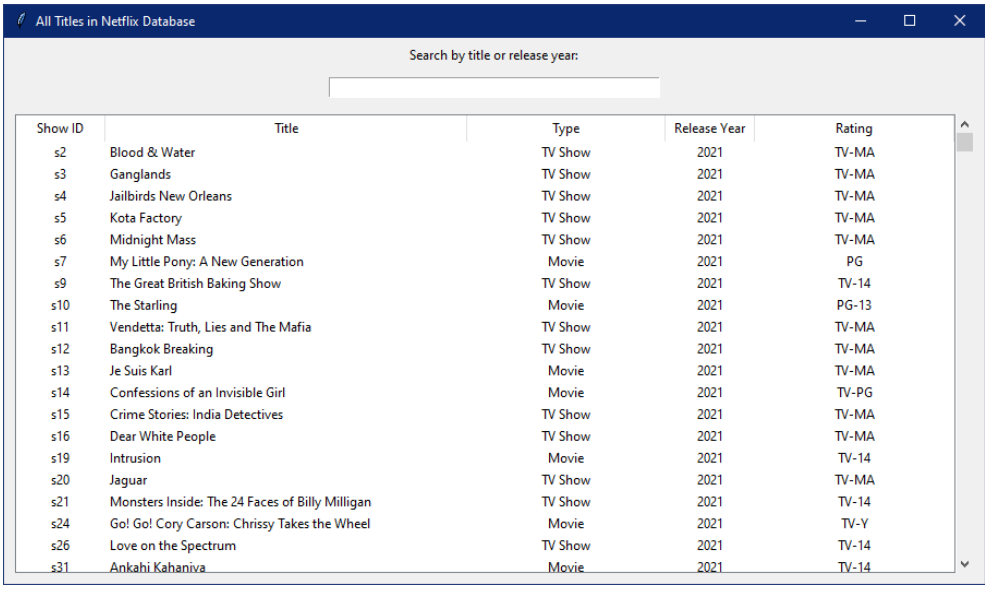
Watchlists

Users can double click on a watchlist to view all titles in the list.

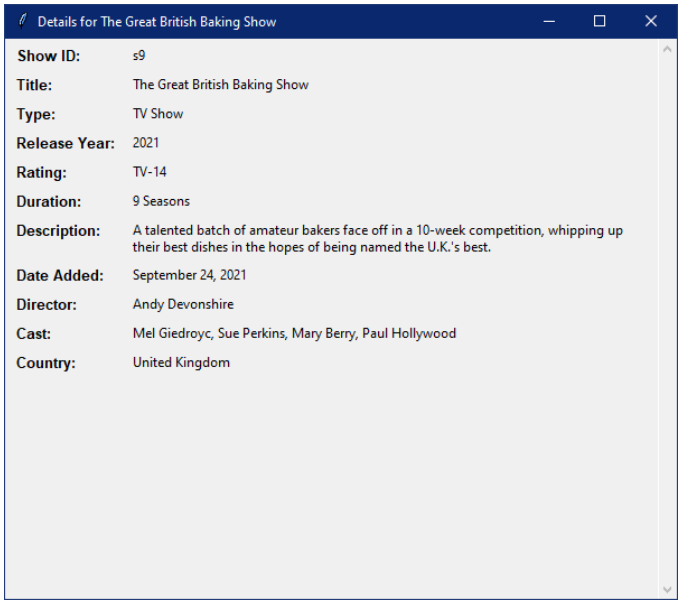


To modify a watchlist, the user can select an already-created watchlist and enter new values for list name and teacher name, then select *Update Selected Watchlist* to update these values.

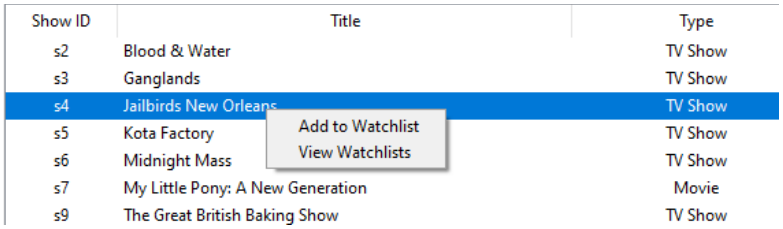
Users can also select *View All Titles* to view all available titles. The user can search by title or release year.



Users can double click on any title to view more info about a title.

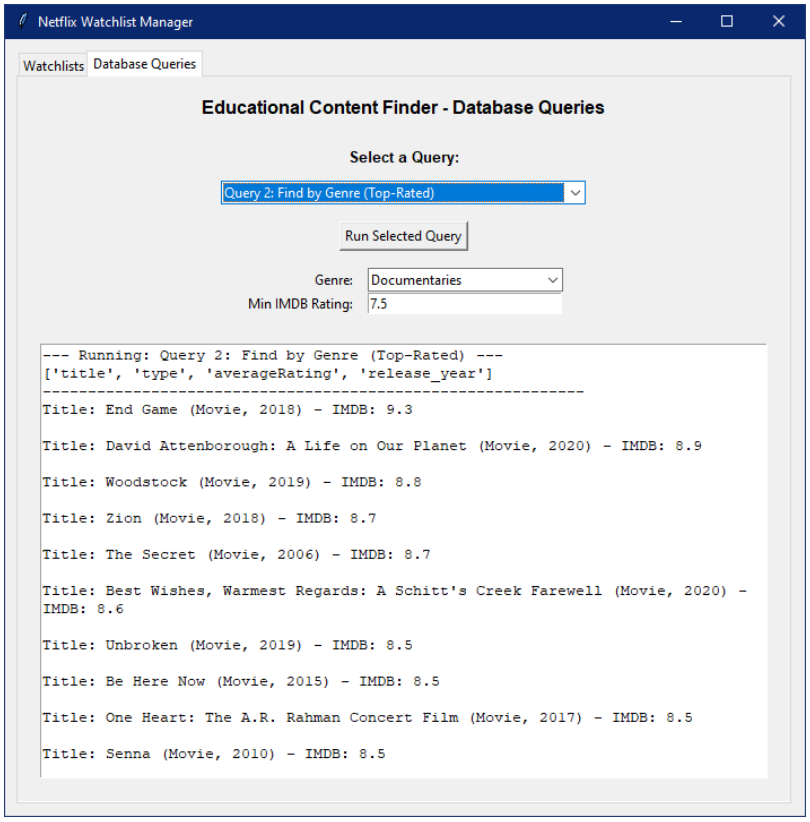


Users can right click on a title to add to a watchlist, or view all watchlists the title is on.



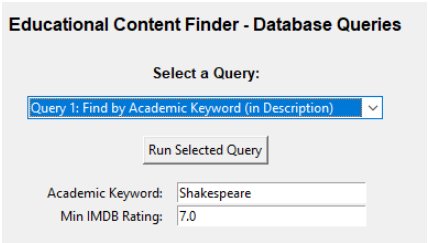
All database operations are delegated to functions imported from *Crud_components.py*, while this file focuses on user interaction, layout, event handling, and dynamic UI updates.

Database Queries

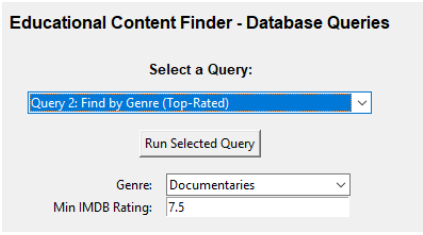


Under the *Database Queries* tab, the user can select from five pre-designed analytical or parameterized queries.

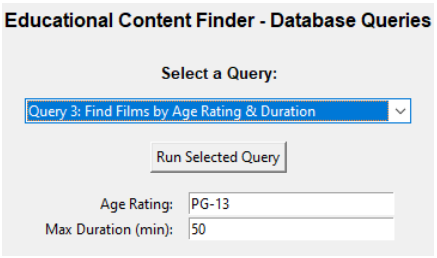
- *Query 1: Find by Academic Keyword (in Description)*
    - The user is able to sort by an academic keyword in the description of a show or movie and enter a minimum IMDB rating to search by.



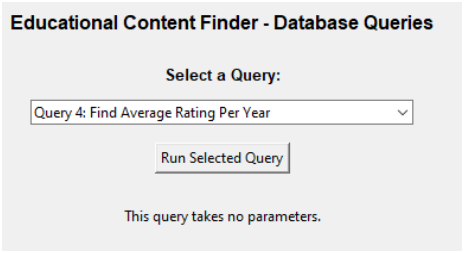
- *Query 2: Find by Genre (Top-Rated)*
    - The user is able to select a genre from a drop-down list of genres and enter a minimum IMDB rating to search by.



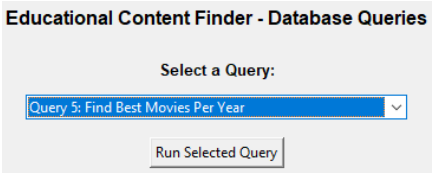
- *Query 3: Find Films by Age Rating and Duration*
    - The user is able to enter an age-rating and a maximum duration in minutes of a show or movie.



- *Query 4: Find Average Rating Per Year*
    - The user is able to view the average rating per year of shows and movies.



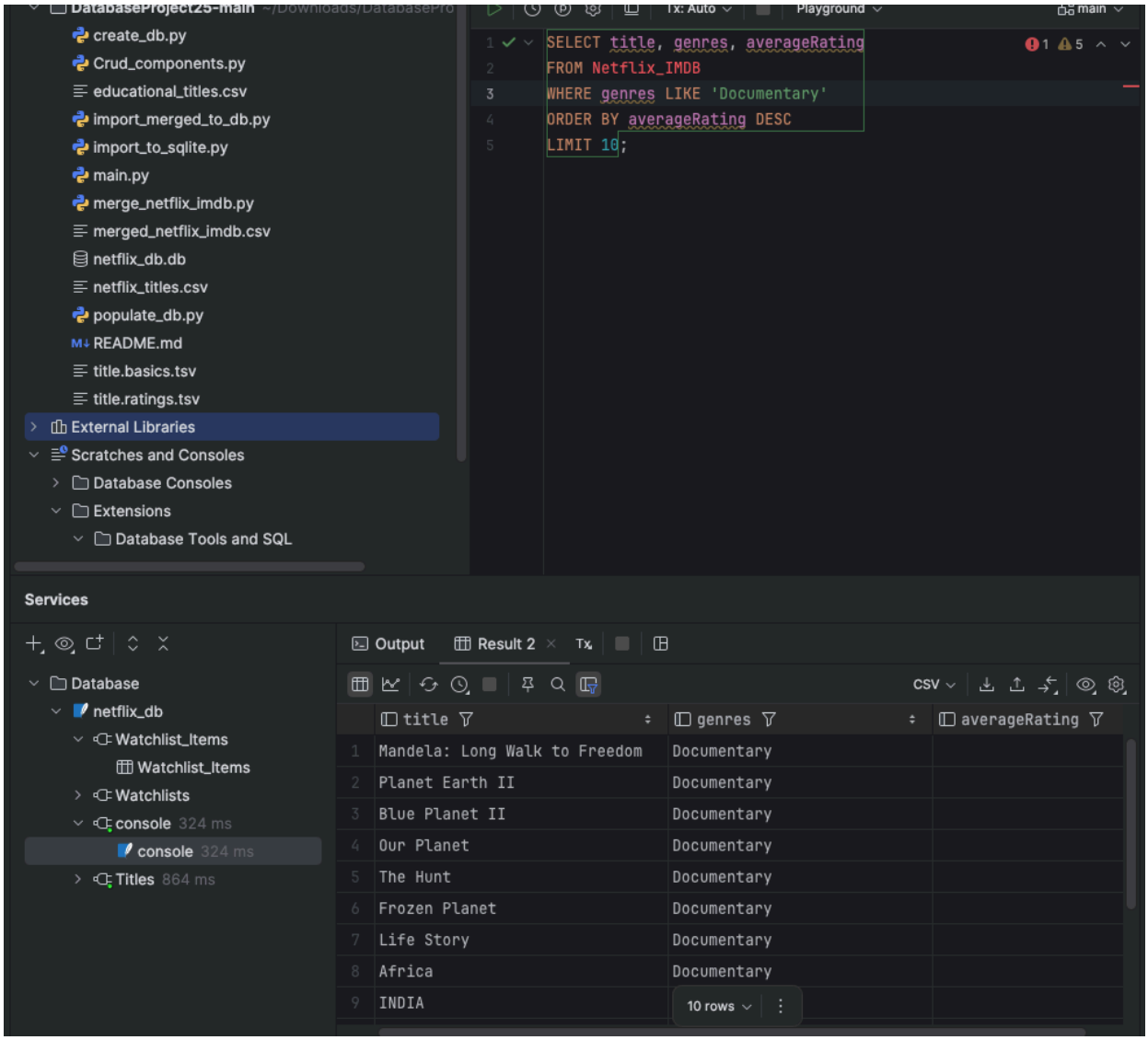
- *Query 5: Find Best Movies Per Year*
    - The user is able to view the highest rated movies per year before 2022.



Data and Documentation

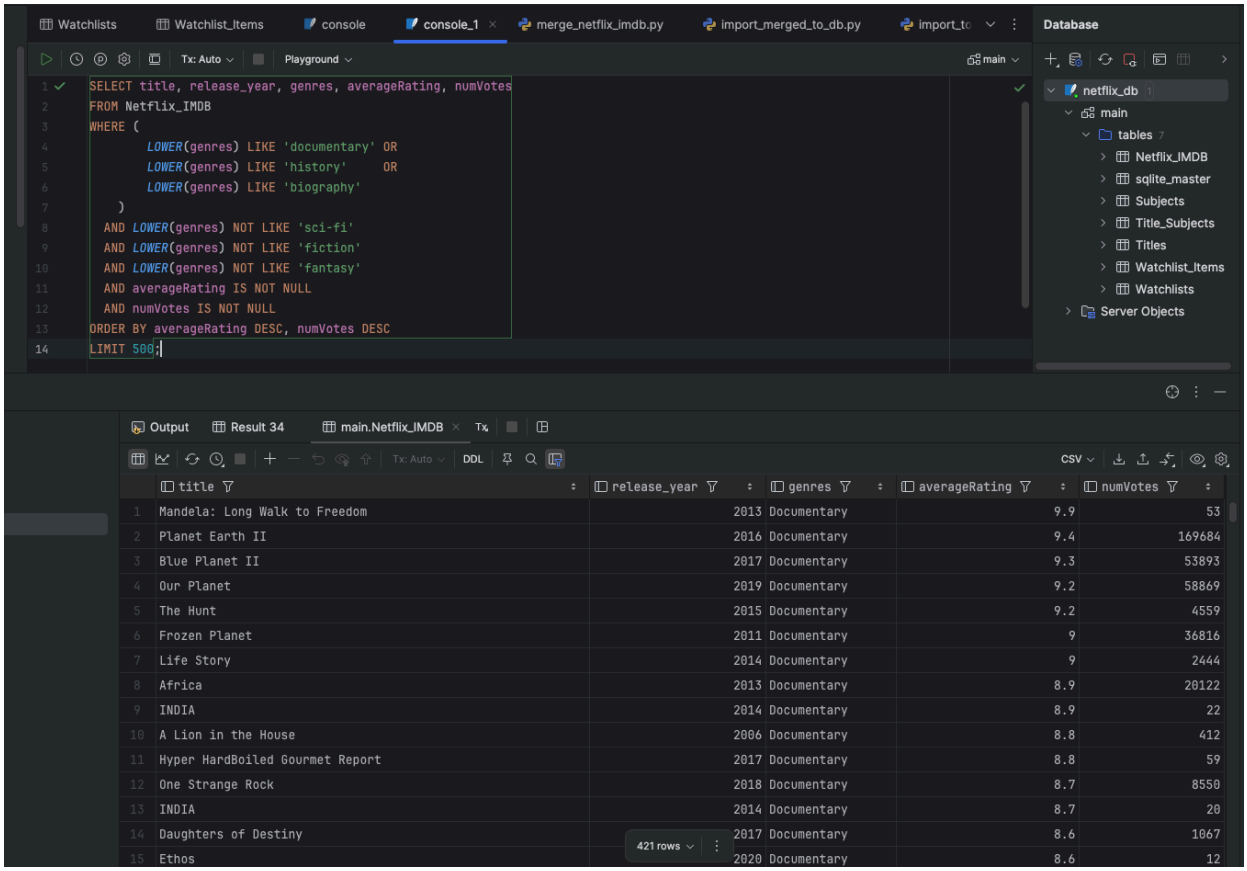
Our database started with the "Netflix_Streaming_Data" dataset from kaggle. This dataset contains over 8500 records with 12 attributes. The attributes included title, type, director, cast, country, release year, rating, duration and category. *(Ahmadrazakashif, n.d.)* The information on

each tv show and movie was very minimal and lacked proper documentation required to identify academically useful content. The image below shows one of the first queries with our original dataset. The limit was 10 but that was also all that was found when the limit was increased. Searching keywords like “Documentary”, “Science”, “History” only brought up over a dozen results.



The IMDb datasets, “title.basics.tsv” and “title.ratings.tsv” were added to the database. *(Internet Movie Database, n.d.)* The results went to over 400 results. The merged datasets were titled “Netflix_IMDB”. The merged dataset linked the descriptions and ratings form IMDB with the

titles in the Netflix dataset. From this point forward we worked on the queries that would help us find the academic and educational content.



	title	release_year	genres	averageRating	numVotes
1	Mandela: Long Walk to Freedom	2013	Documentary	9.9	53
2	Planet Earth II	2016	Documentary	9.4	169684
3	Blue Planet II	2017	Documentary	9.3	53893
4	Our Planet	2019	Documentary	9.2	58869
5	The Hunt	2015	Documentary	9.2	4559
6	Frozen Planet	2011	Documentary	9	36816
7	Life Story	2014	Documentary	9	2444
8	Africa	2013	Documentary	8.9	20122
9	INDIA	2014	Documentary	8.9	22
10	A Lion in the House	2006	Documentary	8.8	412
11	Hyper HardBoiled Gourmet Report	2017	Documentary	8.8	59
12	One Strange Rock	2018	Documentary	8.7	8550
13	INDIA	2014	Documentary	8.7	20
14	Daughters of Destiny	2017	Documentary	8.6	1067
15	Ethos	2020	Documentary	8.6	12

The following queries were added:

Query 1: Keyword + Rating

- Search titles by academic keyword in the description and a minimum IMDb rating.

Query 2: Genre + Rating

- Filter titles by selected genre and a minimum IMDb rating.

Query 3: Age Rating + Duration

- Find films matching an age rating and under a chosen duration.

Query 4: Average Rating per Year

- Show the average IMDb rating for each year.

Query 5: Best Movies per Year

- List the highest-rated movies per year before 2022.

How Our Dataset Is Breaking Barriers

Streaming platforms lack clear labels for academically useful content. Teachers face difficulty finding subject-aligned material. There are also limited search and filtering tools available. These all create barriers to access and academic opportunity. Our database enables fast discovery of high value educational media with our interactive user interface that allows customers to search with a mixture of keywords, genre, rating and title. Our watchlist feature also allows users to organize content selection and can be saved with the teacher's name and class. These features go beyond a standard streaming service search. The Netflix Educational Database improves access to academically useful and educational content. We hope that streaming services can have more robust search and filtering tools in the future.

Reference

Ahmadrazakashif. (n.d.). *Netflix Streaming Data* [Dataset]. Kaggle. https://www.kaggle.com/datasets/ahmadrazakashif/netflix-streaming-data

Internet Movie Database. (n.d.). *IMDb data files* [Data set]. https://datasets.imdbws.com/